
[CS3704] Software Engineering

Dr. Chris Brown

Virginia Tech

9/8/2026

Project Management + Requirements Elicitation

Project Management and Version Control

Requirements Overview

Requirements Engineering

Requirements Elicitation

Learning Outcomes

By the end of the course, students should be able to:

- **Understand software engineering processes, methods, and tools used in the software development life cycle (SDLC)**
- **Use techniques and processes to create and analyze requirements for an application**
- Use techniques and processes to design a software system
- Identify processes, methods, and tools related to phases of the SDLC
- Explain the differences between software engineering processes
- Discuss research questions and current topics related to software engineering
- Create and communicate about the requirements and design of a software application

Course Questions

Syllabus

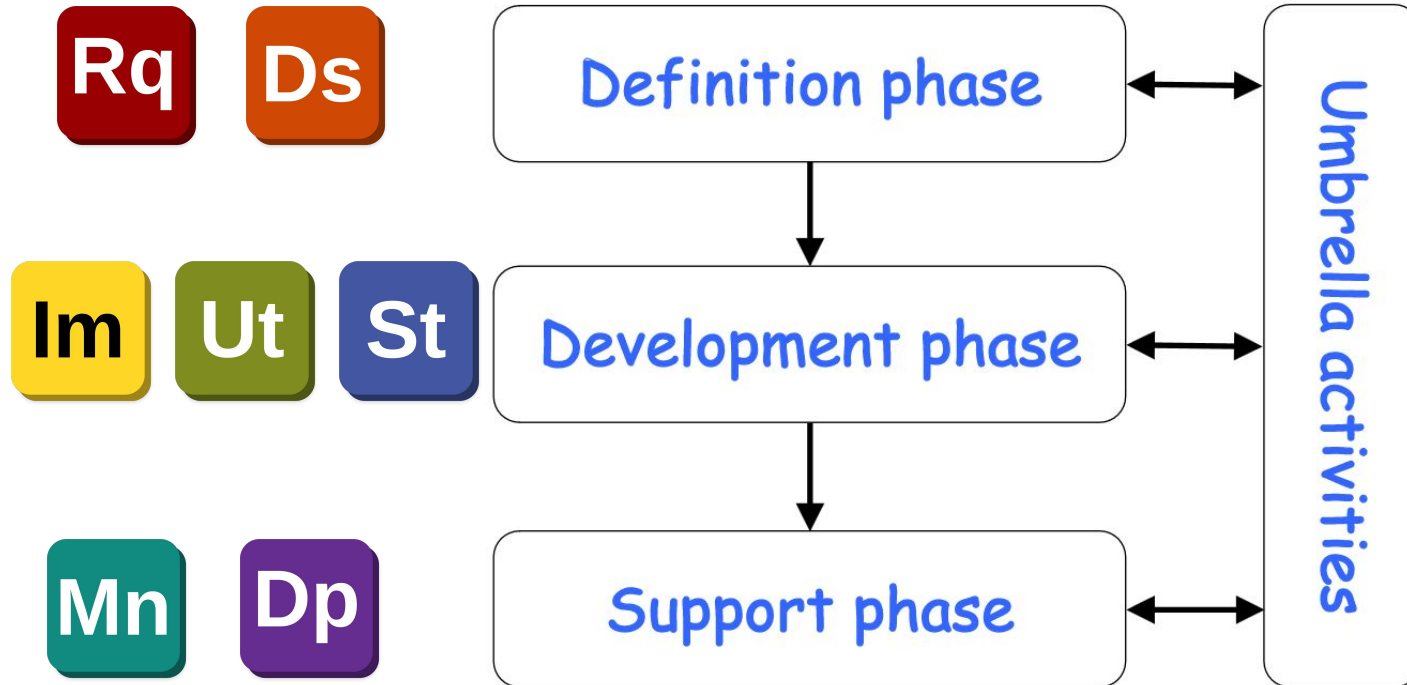
- More specific details on how to setup our computers to work on future assignments. Will we use Docker on VS code? If so, where is the docker file we will use? Are the commands we need to run in HW0.md Software configuration require us to run them in PowerShell/native command line interface for our computers, or on VS code itself? **[Will provide configuration files when needed for assignments and should be able to run configurations natively, however may revisit this based on Project 0/HW0 outcomes]**
- The room number for our class is listed as 240 but is actually 340. Found out the hard way that 240 does not exist on the first day of class **[Sorry!]**
- About the midterm that's online, does that mean we take it out of class or is it still held in-person during class time/The midterm is held online, but will we have to be in class to take this? **[Midterm will be completed outside of class time. I will be traveling for a conference and not present that week so there will be no class.]**
- How the attendance would be checked? **[Usually something submitted on Canvas at the end of class]**
- I have concerns about how much time would the Ut Prosim part take compared to other assignments? **[Depends on selected activities]**
- Bonus: Where will the lectures slides be posted? **[Lecture slides will be posted on the GitHub repo (resources/lectures/) and on Canvas (Modules/Files)]**

Course Questions (cont.)

Project

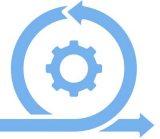
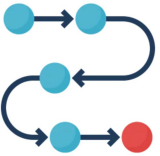
- How would presentations work? [More details later, but presentation will need to provide a motivation of your project (what problem you are trying to solve) and a demo]
- What is the deliverable? Are we creating wireframes or a full application? [Can create a full application or a system prototype (high-fidelity)]
- How much AI can be used when developing the project? [Depends on your team policy, but must use agent-template]
- Is our project allowed to be similar to something that already exists? [Yes, need to explain why]
- How detailed should the github repository be and does the amount of detail affect how our project is evaluated? [Include code, README overview, details on how to run, and issues (bugs or future features)]
- Are there any specific tools required for the project (like docker or other things)? [Only opencode and the agent-template if you are using AI]
- Scope:
 - How big should the scope of the project be if it's due in only a few weeks? / How complex do we want it to be? / What does the scale of the project have to be? How big or small should it be. / How large/small does scope have to be? / How long the project is supposed to be, how detailed etc. Also what would be the difference between first and second project? [Depends on the problem you want to solve and team policy on AI usage]
 - Are there any restrictions for the type of project we choose to do? / Can it literally be anything? (Websites, app, robot, etc) / What type of project should we build? / Are there any specific guidelines or limitations for what our project is allowed to be or what topics we can choose? [No restrictions, within reason]

Review: SE Processes



Review: SE Processes

- **Plan-driven:** Software development is divided into workable increments building on the work completed in the previous increment.
- **Iterative:** Software is developed through repeated, time-boxed, and short cycles (*iterations*).
- **Agile:** Developer-focused iterative and evolutionary development framework.



Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

Project Management

The systematic application of engineering principles, workflows, tools, and team organization to **deliver software predictably and efficiently**.

- It bridges the gap between *what the customer wants* and *how the engineering team builds it* by managing scope, schedules, technical risks, etc.

- Meetings
- Version Control

Agile Project Management

- Agile-Inspired Project Meetings:
 - Stand-ups/Scrum Meeting 
 - Daily <15 min. meetings to provide updates on tasks
 - Sprint planning meetings
 - Triages
 - Sprint review meetings
 - Retrospectives

→ Will differ based on company, development team, etc.

Warm-up

Stand-up Meeting: Since the last stand-up meeting, discuss:

- **What you did.**
- **What you need to do next.**
- **What is blocking you.**

- Standing is optional

Sprint Planning

- Meeting to review and determine tasks to be completed in the upcoming sprint
 - Entire team (Scrum master, manager, developers, etc.)
- Divide work between team members
 - How do you divide work between software engineers? [Wilson]
 - **Feature decomposition**- by feature aspects
 - **Modular decomposition**- by module (i.e. parts of code)
 - **Functional decomposition**- by set of tasks or skill
 - **Rotating decomposition**- by function, but swapping periodically
 - **Chaotic decomposition**- random
- Project Estimation

Sprint Planning: Backlog

The screenshot displays the Visual Studio Backlog interface for the 'Fabrikam Fiber' project. The top navigation bar includes 'Home', 'Code', 'Work', 'Build & Release', and 'Test'. The 'Backlog' tab is selected, showing a list of backlog items. The interface includes a sidebar with 'Epics', 'Features', and 'Backlog items'. The main area shows a table of backlog items with columns for Order, State, Effort, Title, and Tags. The items are categorized into 'Past', 'Current', and 'Future' sprints. The first item is 'Slow response on informatio...' with an effort of 8 and state 'Committed'.

Order	State	Effort	Title	Tags
1	Committed	8	Slow response on informatio...	
2	Committed	5	Change initial view	Web
3	Committed	5	Secure sign-in	Mobile, Web
4	New	3	Welcome back page	Mobile
5	New		Canadian addresses don't display...	RC Review
6	Committed	8	Interim save on long form	
7	New	5	Research architecture changes	Web
8	New	3	Resume	

<https://www.visualstudio.com/en-us/docs/work/backlogs/create-your-backlog>

Sprint Planning: Project Estimation

- Software projects require...
 - Size estimation
 - Effort estimation
 - Time estimation
 - Cost estimation
- But, humans are very bad at estimation...
- **Estimation techniques**
 - Constructive Cost Model (COCOMO) [Boehm]
 - Planning Poker 
 - *Function points*: estimated measure of time, effort, etc. to complete a work task

How can people join the poker room?

Room 44441241

Scan the QR code



<https://www.scrumpoker-online.org/en/room/44441241/scrum-poker>



Copy URL to clipboard



Share via email

Triage Meetings

- Review of bugs and defects
 - Analyze reproducibility
 - Prioritize fixes
 - **NOT** coming up with a solution
- Depending on team, triages can be regularly scheduled, integrated with sprint planning meetings, or only held in case of emergency.

Sprint Review Meeting

- Development team presents work that was completed during the sprint.
 - Often includes a demo of completed features, tasks
 - Collect immediate feedback from stakeholders
- **Anti-patterns** [\[Wolpers\]](#)
 - Powerpoint presentation
 - No clients, stakeholders, or customers in attendance
 - Requirements analysis and specification
 - *Wizard of Oz*'ing
 - Extended scrum or planning meeting
 - Retrospective

Retrospectives

- Review of the sprint to reflect on activities and processes, brainstorm for next sprint.
 - Opportunity for Scrum team to inspect itself **over the last sprint only!**
 - Iterative improvement applied to the team

Retrospective questions:

1. **What went well?**
2. **What didn't go well?**
3. **What should we do differently next time?**

Version Control

Version Control: Software that keeps track of changes to a set of files.

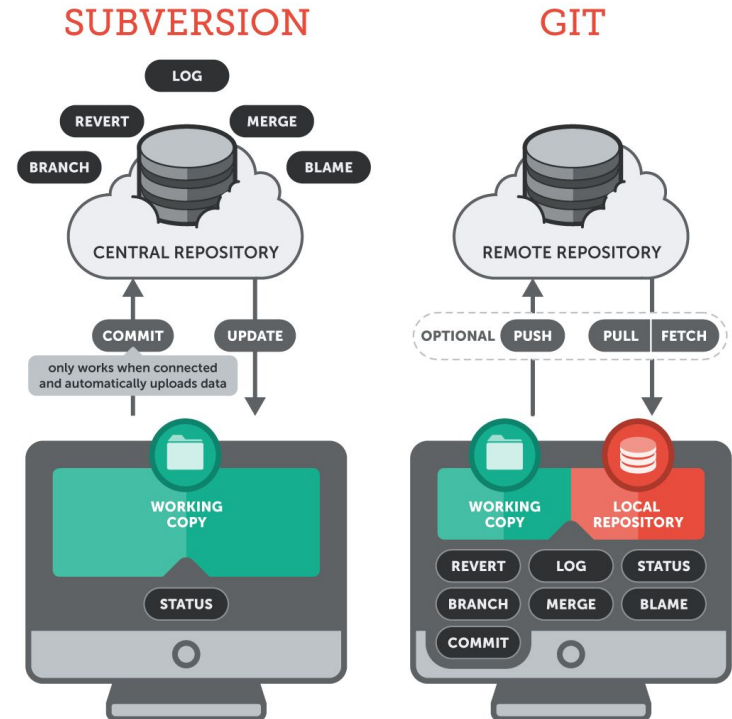
Version control is used all the time outside of code:

- *Microsoft Word*: Ctrl+Z to undo changes (and go back to an earlier version of the document)
- *Google Docs*: can see who made what changes to a file,
- ...

Version Control for SE

Different implementations of version control for software development:

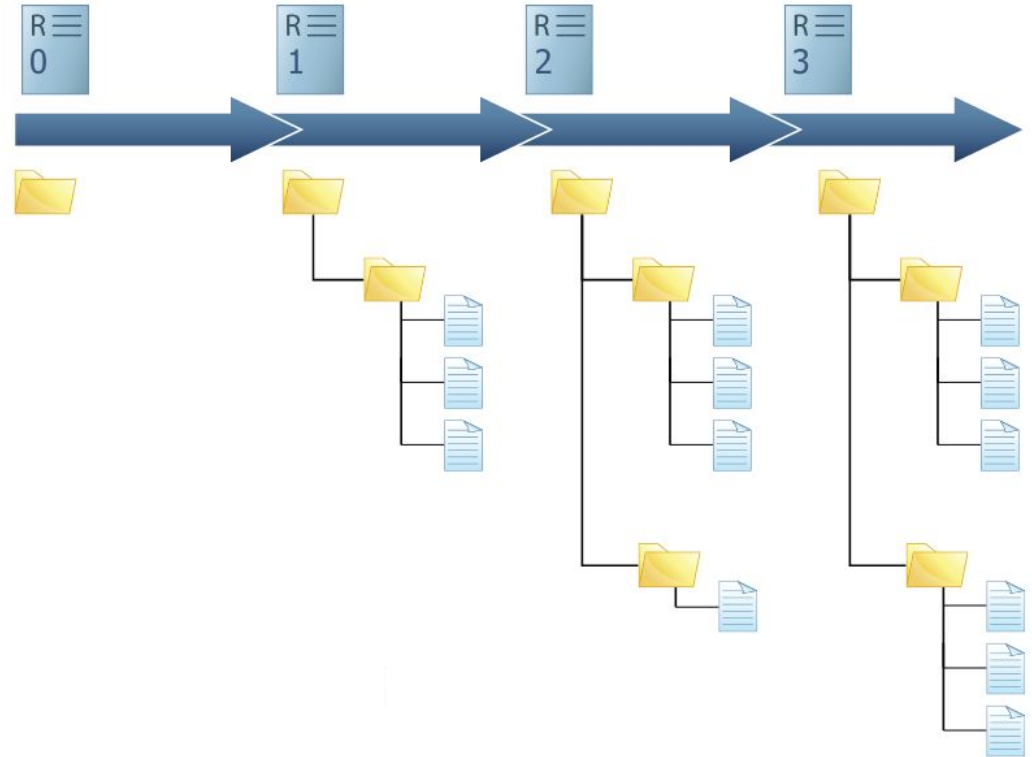
- Git (*i.e.*, GitHub, GitLab, etc.)
- Subversion
- Mercurial



[Git Tower]

Repository Layout

- Each project has multiple revisions
 - Each *revision* is assigned a name
 - Revision number is incremented for every commit transaction
 - *Delta* (diff) information is recorded



Basic Features of a Repository

- Keep the history of all changes to files and directories
 - You can add in new versions
 - You can recover any previous version
 - Access control
 - Read/write permission for users
 - Logging
 - Author, date, and reason for a change
 - Additional features
 - *Diff, Branches, Merging*
-

Basic Workflow

The simplest way to use git is the “linear” workflow:

1. `git init` to enable Git in a certain directory
2. `git add` any files you want Git to “track”
3. `git commit` the currently “staged” changes to save a snapshot
4. make changes to your files
5. `git add` the changed files to “stage” them again
6. Repeat #3 (`git commit`)
7. Retrieve changes with `git pull`

You can use `git log` to see your commit history, and `git status` to see the current state of staged/unstaged changes.

GIT: FOUR PHASES WITH REMOTE

**Working
Directory**

**Staging
Area/Index**

Local Repository

**Remote
Repository**

`git stage ...`
`/git add`

`git commit`

`git push`

`git pull`

Branching Workflow

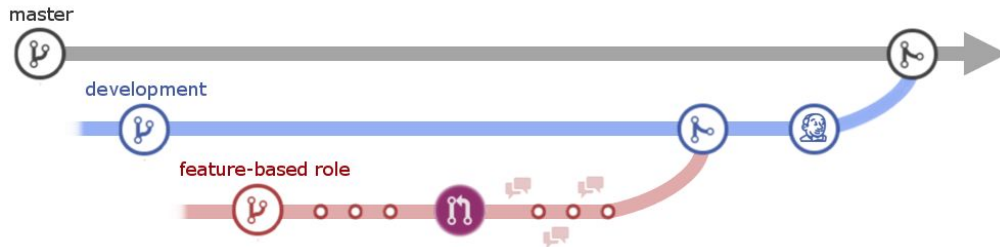
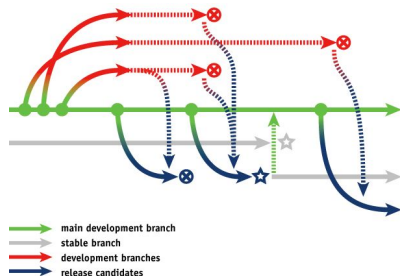
You can split repos into branches, allowing people can work on different things without interfering with one another.

1. Make sure your repository is “clean” (i.e., no uncommitted changes).
 2. `git checkout -b` to create a new branch and move to it; at this point, the new branch is identical to the old one.
 3. Make changes, `git add`, `git commit` as usual (linear flow)
 4. `git checkout` to switch between branches
 5. Merging!
-

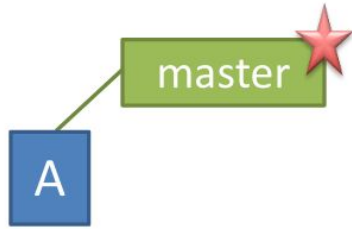
Branching

A **branch** is a mechanism for allowing concurrent changes to be made to a source repository.

- Branching is effective, but developers create ineffective branches. [Bird]

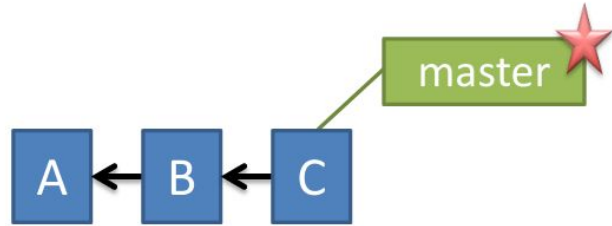


Basic Merge Example



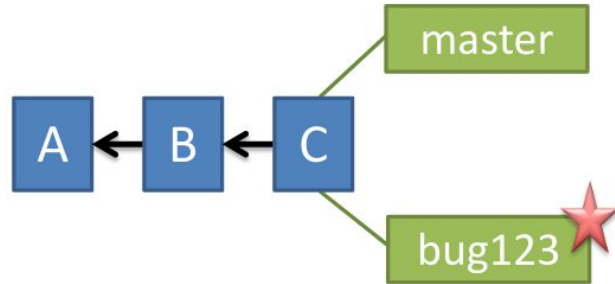
```
> git commit -m 'my first commit'
```

Basic Merge Example



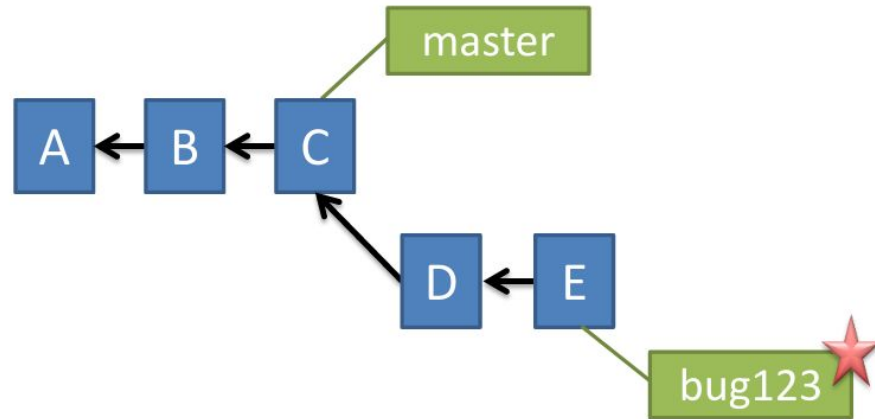
```
> git commit (x2)
```

Basic Merge Example



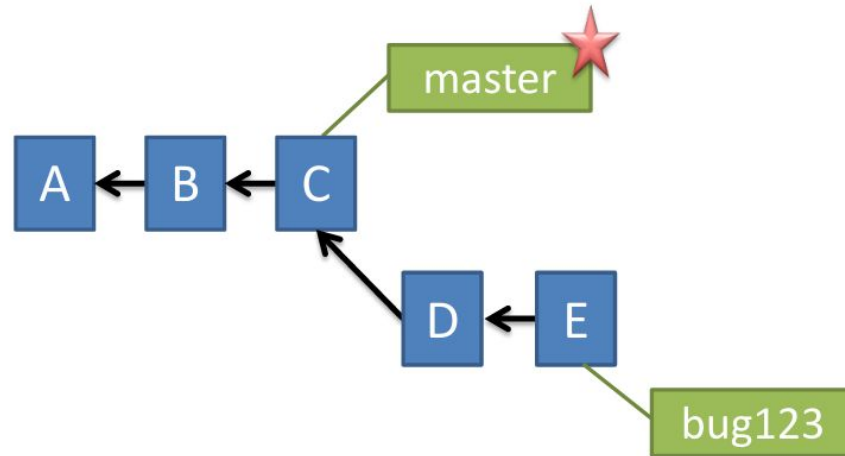
```
> git checkout -b bug123
```

Basic Merge Example



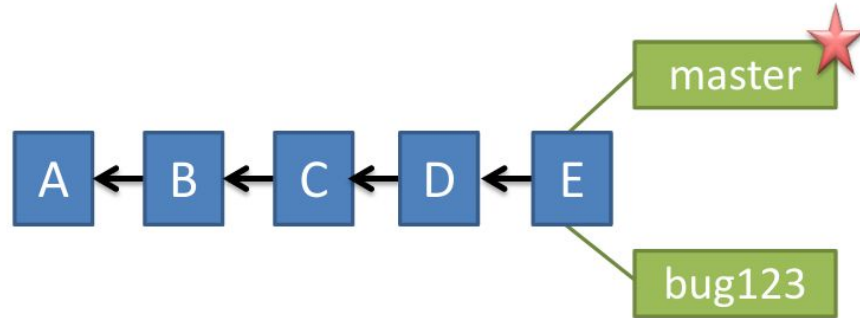
```
> git commit (x2)
```

Basic Merge Example



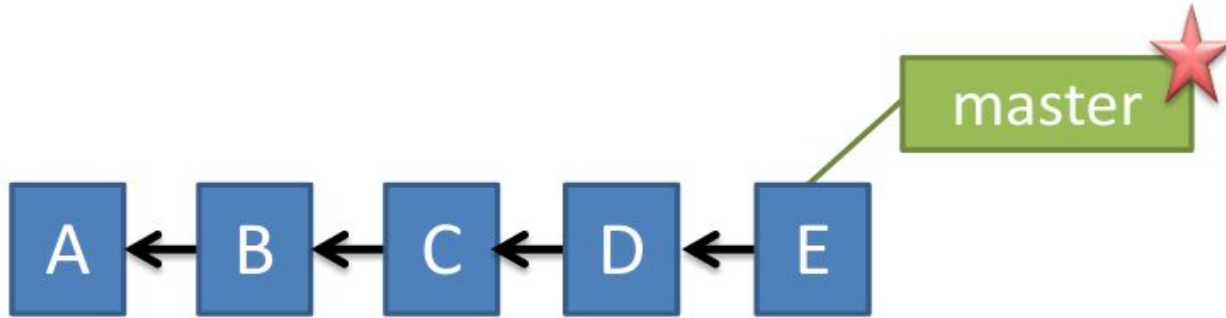
```
> git checkout master
```

Basic Merge Example



```
> git merge bug123
```

Basic Merge Example



Merge Conflicts

- When two developers edit the same code
 - Can be resolved manually or automatically

```
void f(int i) {  
    <<<<<<<< .mine  
    int j = 3;  
    =====  
    int j = 4;  
    >>>>>>>> .rl3
```

Tips:

- Small commits
- Commit often!
- Write meaningful commit messages

TABLE 5
Measures to Estimate the Difficulty/Time
to Resolve Merge Conflicts

Measure	#Sug.
Number of conflicting lines of code (<i>#ConfLOC</i>)	19
Number of conflicting chunks (<i>#ConfChunks</i>)	16
Number of lines of code changed (<i>#LOC</i>)	13
Number of files changed (<i>#Files</i>)	9
Time between the base commit and the merge commit	5
Developer experience responsible for conflicting changes ($\sim \%IntegratorKnowledge$)	4
Number of conflicting files (<i>#ConfFiles</i>)	4
Frequency target file changed	4
Semantically diff between conflicting code	4
Number of active developers (<i>#Devs</i>)	3
Number of commits with conflicts	3
Developer knowledge on the project ($\sim \%IntegratorKnowledge$)	3
Number of callers and callees functions in the conflicting code	3

Pull Requests

In reality, it's rare to merge branches locally and push them to remote. Instead, we use services like GitHub or GitLab to help us:

- Step 1: Create a fork (your own copy)
- Step 2: Clone repo locally
- Step 3: Create branch and make some commits
- Step 4: Push those commits to your branch
- Step 5: Open a pull/merge request
- Step 6: Collaborate, leave comments, fix issues, etc. (code reviews)
- Step 7: Merged into main (or other branch) by maintainer

Version Control System handles branching and merging under the hood.

Requirements

Rq

Goal: Understand customer requirements for the software system

- The *what* of the project
- Very difficult to “get right” the first time and evolve over the course of development
 - Remember the Top 3 reasons for project failure:
(2) Incomplete and (3) Changing Requirements
- *Software Artifacts*: stakeholder insights, survey/interview responses, elicitation results...

What are requirements?

Definition: Capabilities and conditions to which the system — and more broadly, the project — must conform. [Larman]

- Focusing on the **WHAT** not the **HOW**
- **Should always come first in the SDLC**

Types of Requirements

- **Functional:** expresses a function that an application must perform
 - Example: “HokieSpa must read students academic course data for students to view progress towards their degree”
- **Nonfunctional:** does not involve specific functionality, but should be specific, quantifiable, and testable

Non-functional Requirements

Examples:

- **Usability:** human factors, help documentation, etc.
 - “HokieSPA must meet **WCAG 2.1 AA accessibility standards**, ensuring compatibility with screen readers.”
- **Reliability:** failure frequency and recoverability
 - “In case of system crash, HokieSPA should recover to an operational state **within 5 minutes**.”
- **Performance:** response times, throughput, availability, resource usage, etc.
 - “Page load time for any core task (i.e., registration) should not exceed **2 seconds** on campus Wi-Fi.”

Non-functional Requirements (cont.)

- **Supportability:** adaptability, maintainability, internationalization, configurability
 - “The system should **detect and log 100% of critical errors** for review by IT staff.”
- **Implementation/Constraints:** resource limitations, languages, tools, hardware
 - “HokieSPA must be written in **.NET** on a **Linux server**.”

Why should we care?

“The hardest single part of building a software system is deciding precisely what to build. No other part of the conceptual work is so difficult as establishing the detailed technical requirements, including all the interfaces to people, to machines, and to other software systems. No other part of the work so cripples the resulting system if done wrong. No other part is more difficult to rectify later.”

- Fred Brooks



E-mail: SCOTTADAMS@AOL.COM



© 2006 Scott Adams, Inc. /Dist. by UFS, Inc.



P-27-04



<http://dilbert.com/stip/2006-01-29>

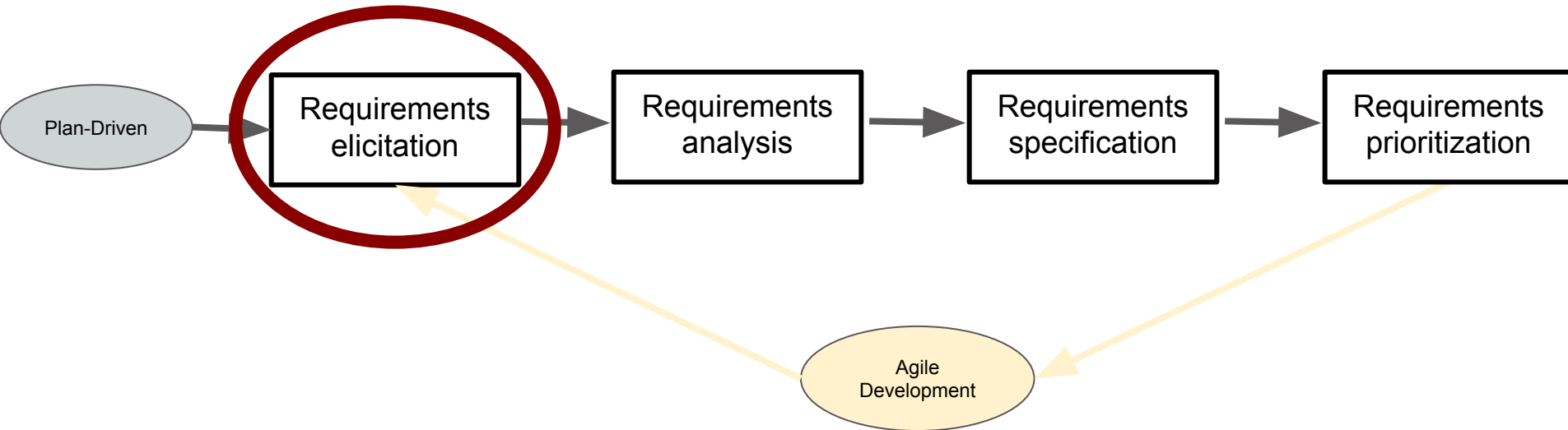
Requirements Engineering

- **Definition:** the a set of activities concerned with identifying and communicating the purpose of a software-intensive system, and the contexts in which it will be used. Hence, RE acts as the bridge between the real-world needs of users, customers, and other constituencies affected by a software system, and the capabilities and opportunities afforded by software-intensive technologies. [Easterbrook]

Requirements Engineering

The process of defining, understanding, documenting, and maintaining requirements.

- Subset of software engineering
- Introduced for the waterfall model (first phase)



Requirements Elicitation

- The process of identifying business needs, scope, assumptions, and risks of a project based on data from key stakeholders.
 - This is the main way requirements are identified.
 - Increases the likelihood users get what they want, and reduces the risk of project failure.

Requirements Elicitation Steps

1. Gathering requirements
2. Identifying key stakeholders
3. Eliciting requirements from key stakeholders
4. Documenting requirements
5. Confirming the findings

Requirements Elicitation Steps

1. Gathering requirements
2. Identifying key stakeholders
3. Eliciting requirements from key stakeholders
4. Documenting requirements
5. Confirming the findings

Questions to ask

Questions to ask internally:

- What is the project objective?
- What problem are you trying to solve?
- What is the success criteria?
- Who is the target audience?
- What other stakeholders will be impacted by the system?
- What is the approval process for requirements?
- ...

Requirements Elicitation Steps

1. Gathering requirements
2. Identifying key stakeholders
- 3. Eliciting requirements from key stakeholders**
4. Documenting requirements
5. Confirming the findings

Elicitation Techniques

Requirements Gathering

aka discovering the requirements of a system

→ Traditional techniques

- ↳ Introspection
- ↳ Reading existing documents
- ↳ Analyzing hard data
- ↳ Interviews
 - Open-ended
 - Structured
- ↳ Surveys / Questionnaires
- ↳ Meetings

→ Collaborative techniques

- ↳ Focus Groups
 - Brainstorming
 - JAD/RAD workshops
- ↳ Prototyping
- ↳ Participatory Design

→ Contextual (social) approaches

- ↳ Ethnographic techniques
 - Participant Observation
 - Ethnomethodology
- ↳ Discourse Analysis
 - Conversation Analysis
 - Speech Act Analysis
- ↳ Sociotechnical Methods
 - Soft Systems Analysis

→ Cognitive techniques

- ↳ Task analysis
- ↳ Protocol analysis
- ↳ Knowledge Acquisition Techniques
 - Card Sorting
 - Laddering
 - Repertory Grids
 - Proximity Scaling Techniques

Types of Data to Collect

- Quantitative
 - Can be objectively measured and counted
- Qualitative
 - Cannot be objectively measured or counted
- Categorical
 - Mix of both quantitative and qualitative
- Demographic
 - Background details of participants

→ ***Ideally, you should collect all of them!***

Questions to Ask

Questions to ask externally:

- What problems do you face that this system should solve?
- What are your primary goals when using the system?
- How do you currently perform this task or process?
- What do you like or dislike about the current system/process?
- What are the most frustrating aspects of the current system/process?
- How easy is it to use the current system on a scale of 1 to 10? Why?
- How often do you use the system (daily, weekly, monthly)?
- What would make your work easier or more efficient?
- What specific features/functions would you like to see in the new system?
- How do you see your work or industry changing over the next few years?
- What is the approval process for requirements?
- ...

Elicitation Techniques (Surveys)



- Advantages

- Good during the early stages of requirements elicitation
- Can incorporate multiple types of questions and data (i.e., open-ended or closed-ended questions)
- More efficient way to collect information from a larger group

- Disadvantages

- Questions must be well-formulated to avoid redundancy and irrelevance, misunderstandings
- Limited depth of knowledge acquired

Structuring Surveys



If conducting a survey, balance between **closed-ended questions** (to quantify user preferences) **open-ended questions** (to gather detailed qualitative insights).

- **Closed-ended question example:**

- Likert Scale
 - How satisfied are you with the current system? [1 = very dissatisfied, 5 = very satisfied]
 - 👍 👎
 - 😊 😐 😡 😞
- True or False
- Yes/No

- **Open-ended question example:**

- Free response questions
 - What are the top three features you would like to see in the new system and why?

Elicitation Techniques (Interviews)



- **Advantages**
 - Rich collection of information
 - Good for uncovering opinions, feelings, goals, as well as hard facts
 - Can probe in depth and adapt follow-up questions to what the person tells you [can be structured or unstructured]
- **Disadvantages**
 - Time-consuming
 - Large amount of qualitative data can be hard to analyze
 - Hard to compare different respondents
 - Interviewing is a difficult skill to master

Elicitation Techniques (Others)

Group Methods: Brainstorming, Focus Groups



- Advantages: More natural interaction between people than formal interview
- Disadvantages: Biased groups, “Group-think”, Highly trained facilitator

Task-based Methods: Prototyping



- Advantages: Contextualized, Visual aids
- Disadvantages: What you see is what you get (WYSIWYG), Define tasks

Ethnography: Observation in a natural setting



- Advantages: Highly contextualized, Reveals details other methods cannot
- Disadvantages: Extremely time consuming, Hard to analyze

Requirements Elicitation Steps

1. Gathering requirements
2. Identifying key stakeholders
3. Eliciting requirements from key stakeholders
4. Documenting requirements
5. Confirming the findings

Questions to ask (cont.)

Questions to ask yourself (iteratively):

- What are the key functions or features the system should provide?
- What are the "must-have" vs. "nice-to-have" features?
- Are there specific deadlines or timeframes for delivery?
- ...

Non-functional Requirements and Risk Analysis

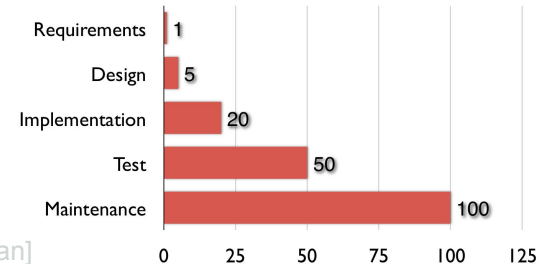
- What performance standards are expected (e.g., response time)?
- Are there any security requirements?
- What data needs to be captured, stored, and processed?
- Does the system need to integrate with any other systems or tools?
- What support or maintenance will be required post-launch?
- What are the potential risks to the project's success?
- ...

Requirements Elicitation Difficulties

Requirements elicitation is challenging, *but **critical*** for software teams

What Happens When You Skip Elicitation?

- **Building the Wrong Product:** You spend months building a system that does not address the actual business problem.
- **Rework:** Fixing a requirements error during the testing or maintenance phase costs *up to 100 times more* than getting it right during elicitation.
- **Scope Creep:** Without strict requirements and stakeholder input, project features can expand chaotically during development.
- **Reminder:** The CHAOS report showed poor user input and incomplete requirements were the top reasons for project failures!



[Pressman]

Requirements Elicitation and AI

There is considerable optimism over LLMs for code generation, but what about other SDLC tasks?

1. AI for Generating Elicitation
2. AI for Analyzing Elicitation Results

AI for Generating Elicitation

AI can be used for generating requirements elicitation questions.

LLMREI: Automating Requirements Elicitation Interviews with LLMs
Alexander Korn, Samuel Gorsch, Andreas Vogelsang

⚙️ System Prompt: LLMREI-short

You are an interviewer, called LLMREI, who assists a requirements engineer in eliciting requirements. Bombard the stakeholder with questions about his/her business and his/her project to find out everything the stakeholder envisions! Act like a real-world interviewer, so only ask one question at a time or only ask two questions if it is about one specific topic.

- *Captured 70% of intended requirements.*

AI for Analyzing Results

LLMs are great at annotating data 



- Transcription,
- Summarization,
- Analysis,
- etc...



Class Activity

TODO: With your Project 1 group, complete the following:

1. Create an initial mechanism for requirements elicitation for your project idea (i.e., interview questions, survey, tasks, etc.). Include at least 5 questions.
2. When you finish, swap with another group to get feedback on your questions for eliciting requirements for your system. Feedback could include but is not limited to: What could be improved? What questions are missing? Wording,...
3. Submit the following on Canvas [this will be graded for attendance]:
 - a. The method you selected for requirements elicitation and why.
 - b. The name/members of the team you swapped with. ← for attendance!
 - c. A summary of the feedback on your preliminary devised questions.
 - d. [If you used AI] The AI tool, prompt, and a review of the generated questions.

Due: at the end of the day (submit whatever you complete)

*** Note:** *You will need to do requirements elicitation for Project Milestone 1.1!*

**** Note:** *No, this does not count towards the 10 required responses for PM1.1!*

Next Class...

- **Requirements Analysis and Specification**

Announcements

- **HW1 due 9/11 at 11:59pm**
- **PM1.1 due 9/16 at 11:59pm**

References

- Mohammadmehdi Ataei, Hyunmin Cheong, Daniele Grandi, Ye Wang, Nigel Morris, and Alexander Tessier. "Elicitron: A large language model agent-based simulation framework for design requirements elicitation." Journal of Computing and Information Science in Engineering. Feb 2025.
- Toufique Ahmed, Premkumar Devanbu, Christoph Treude, and Michael Pradel. 2024. Can LLMs replace manual annotation of software engineering artifacts? arXiv preprint arXiv:2408.05534 (2024)
- Jeffrey S Bowers, Guillermo Puebla, Sushrut Thorat, Konstantinos Tsetsos, and Casimir Johannes Hendrikus Ludwig. Centaur: A model without a theory, July 2025. URL https://osf.io/v9w37_v3.
- Korn, Alexander, Samuel Gorsch, and Andreas Vogelsang. "LLMREI: Automating Requirements Elicitation Interviews with LLMs." arXiv preprint [arXiv:2507.02564](https://arxiv.org/abs/2507.02564) (2025).
- Sarah Schröder, Thekla Morgenroth, Ulrike Kuhl, Valerie Vaquet, and Benjamin Paaßen. "Large Language Models Do Not Simulate Human Psychology". arXiv preprint [arXiv:2508.06950](https://arxiv.org/abs/2508.06950) (2025).
- Igor Steinmacher, Jacob Mcauley Penney, Katia Romero Felizardo, Alessandro F. Garcia, and Marco A. Gerosa. "Can ChatGPT emulate humans in software engineering surveys?." In Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, pp. 414-419. 2024.
- Git Tower. "Switching from Subversion to Git". <https://www.git-tower.com/learn/git/ebook/en/command-line/appendix/from-subversion-to-git>
- Steve Easterbrook (University of Toronto)
- Sarah Heckman (NC State)
- Na Meng and Barbara Ryder (VT)